

ECC

AIM

To implement **Elliptic Curve Cryptography (ECC)** using ECDH to generate a secure shared secret between two users.

ALGORITHM

1. Start the program.
2. Generate an ECC key pair for **Alice**.
3. Generate an ECC key pair for **Bob**.
4. Exchange the public keys.
5. Compute the shared secret using each user's private key and the other's public key.
6. Compare the generated shared secrets.
7. Display the shared secret and verify that both are equal.
8. Stop the program.

PROGRAM

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.util.Arrays;
import javax.crypto.KeyAgreement;

public class Main {
    public static void main(String[] args) throws Exception {

        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("EC");
        keyGen.initialize(256);

        KeyPair alicePair = keyGen.generateKeyPair();
        KeyPair bobPair = keyGen.generateKeyPair();

        KeyAgreement alice = KeyAgreement.getInstance("ECDH");
```

```

    alice.init(alicePair.getPrivate());

    alice.doPhase(bobPair.getPublic(), true);

    byte[] aliceSecret = alice.generateSecret();


    KeyAgreement bob = KeyAgreement.getInstance("ECDH");

    bob.init(bobPair.getPrivate());

    bob.doPhase(alicePair.getPublic(), true);

    byte[] bobSecret = bob.generateSecret();


    System.out.println("Alice Shared Secret:");

    for (byte b : aliceSecret)

        System.out.printf("%02X", b);


    System.out.println("\n\nBob Shared Secret:");

    for (byte b : bobSecret)

        System.out.printf("%02X", b);

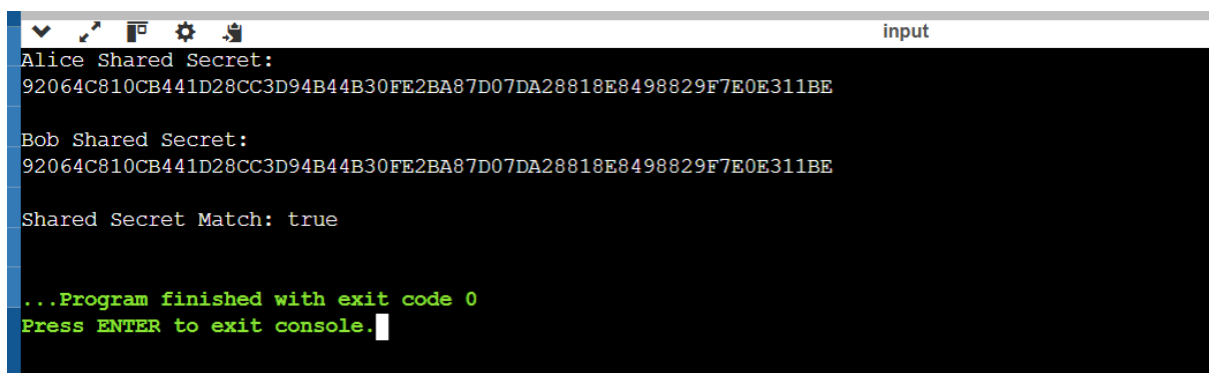

    System.out.println("\n\nShared Secret Match: " +

        Arrays.equals(aliceSecret, bobSecret));

}

}

```



```

input
Alice Shared Secret:
92064C810CB441D28CC3D94B44B30FE2BA87D07DA28818E8498829F7E0E311BE

Bob Shared Secret:
92064C810CB441D28CC3D94B44B30FE2BA87D07DA28818E8498829F7E0E311BE

Shared Secret Match: true

...Program finished with exit code 0
Press ENTER to exit console.

```

RESULT

Thus, **ECC using ECDH** was implemented successfully, and both users generated the same shared secret.

DIFFIE-HELLMAN

AIM

To implement the **Diffie-Hellman key exchange algorithm** to securely generate a shared secret key between two users.

ALGORITHM

1. Start the program.
2. Generate the key pair for **Alice**.
3. Generate the key pair for **Bob**.
4. Exchange their public keys.
5. Compute the shared secret using the private key and received public key.
6. Compare both generated shared secrets.
7. Display the shared secret and verify that both are equal.
8. Stop the program.

PROGRAM:

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.util.Arrays;
import javax.crypto.KeyAgreement;

public class Main {

    public static void main(String[] args) throws Exception {

        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("DH");
        keyGen.initialize(2048);

        KeyPair alicePair = keyGen.generateKeyPair();
        KeyPair bobPair = keyGen.generateKeyPair();

        KeyAgreement alice = KeyAgreement.getInstance("DH");
```

```

    alice.init(alicePair.getPrivate());
    alice.doPhase(bobPair.getPublic(), true);
    byte[] aliceSecret = alice.generateSecret();

    KeyAgreement bob = KeyAgreement.getInstance("DH");
    bob.init(bobPair.getPrivate());
    bob.doPhase(alicePair.getPublic(), true);
    byte[] bobSecret = bob.generateSecret();

    System.out.println("Alice Shared Secret:");
    for (byte b : aliceSecret)
        System.out.printf("%02X", b);

    System.out.println("\n\nBob Shared Secret:");
    for (byte b : bobSecret)
        System.out.printf("%02X", b);

    System.out.println("\n\nShared Secret Match: " +
        Arrays.equals(aliceSecret, bobSecret));
}
}

```

```
input
Alice Shared Secret:
90421CA42C766C01B1AD7F47C1F839AF862123B4C23C6D20978B94407AD9A18C59F5E043CF09A6911D03B6C79155305D2E8AD7A7A64F523D04DC1853A508B59A0B8C25F85
3951F77581AA49CA5923E16A518A2FCB6AF4A427BC1FEDA5399AD755A0E86AD9BA851829D5B1EE8E482BC109C07EDD94ED7B0C955F87D37ED37900457EFEA86936B36136C
8599D1EFCDE7069CF170B2B308EAF83DA27ED5C6560D160FC82AF1EA74B2F92B3D5203AF54C18CCD2CA9114FB64A47647C6348E83090BD74B472CE1A5853E791ED5671E8F
B87E929538EB2225B67D349A67CADA30904A65A66E373740617371F711A35B60879D5A1436E1147DB6124242D91AFE000121E

Bob Shared Secret:
90421CA42C766C01B1AD7F47C1F839AF862123B4C23C6D20978B94407AD9A18C59F5E043CF09A6911D03B6C79155305D2E8AD7A7A64F523D04DC1853A508B59A0B8C25F85
3951F77581AA49CA5923E16A518A2FCB6AF4A427BC1FEDA5399AD755A0E86AD9BA851829D5B1EE8E482BC109C07EDD94ED7B0C955F87D37ED37900457EFEA86936B36136C
8599D1EFCDE7069CF170B2B308EAF83DA27ED5C6560D160FC82AF1EA74B2F92B3D5203AF54C18CCD2CA9114FB64A47647C6348E83090BD74B472CE1A5853E791ED5671E8F
B87E929538EB2225B67D349A67CADA30904A65A66E373740617371F711A35B60879D5A1436E1147DB6124242D91AFE000121E

Shared Secret Match: true

...Program finished with exit code 0
Press ENTER to exit console.
```

RESULT

Thus, the **Diffie-Hellman key exchange algorithm** was implemented successfully, and both users generated the same shared secret key.

EX. NO: 6

IMPLEMENTATION OF DES

AIM:

To write a C program to implement Data Encryption Standard (DES) using C Language.

ALGORITHM:

STEP-1: Read the 64-bit plain text.

STEP-2: Split it into two 32-bit blocks and store it in two different arrays.

STEP-3: Perform XOR operation between these two arrays.

STEP-4: The output obtained is stored as the second 32-bit sequence and the original second 32-bit sequence forms the first part.

STEP-5: Thus the encrypted 64-bit cipher text is obtained in this way. Repeat the same process for the remaining plain text characters.

PROGRAM:

```
import java.util.Scanner;

import java.security.SecureRandom;

import javax.crypto.Cipher;

import javax.crypto.KeyGenerator;

import javax.crypto.SecretKey;

import javax.crypto.spec.SecretKeySpec;

public class Main {

    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");

        String message = sc.nextLine();
```



```
KeyGenerator keyGen = KeyGenerator.getInstance("DES");
SecureRandom random = SecureRandom.getInstance("SHA1PRNG");
random.setSeed("12345".getBytes());

keyGen.init(56, random);
SecretKey secretKey = keyGen.generateKey();

byte[] raw = secretKey.getEncoded();

System.out.print("DES Key: ");
for (byte b : raw)
    System.out.printf("%02X", b);
System.out.println();

Cipher cipher = Cipher.getInstance("DES");

cipher.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(raw, "DES"));
byte[] encrypted = cipher.doFinal(message.getBytes());

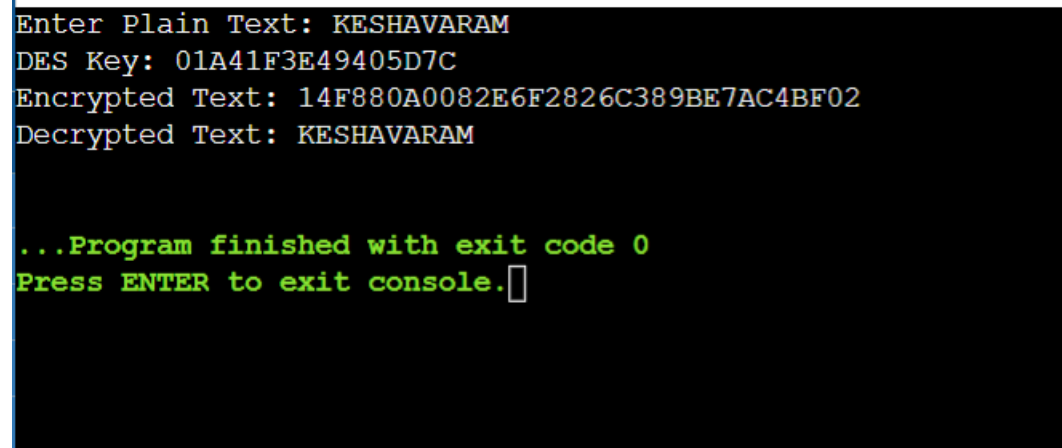
System.out.print("Encrypted Text: ");
for (byte b : encrypted)
    System.out.printf("%02X", b);
System.out.println();

cipher.init(Cipher.DECRYPT_MODE, new SecretKeySpec(raw, "DES"));
byte[] decrypted = cipher.doFinal(encrypted);

System.out.println("Decrypted Text: " + new String(decrypted));
```

```
        sc.close();  
    }  
}
```

OUTPUT:



```
Enter Plain Text: KESHAVARAM  
DES Key: 01A41F3E49405D7C  
Encrypted Text: 14F880A0082E6F2826C389BE7AC4BF02  
Decrypted Text: KESHAVARAM  
  
...Program finished with exit code 0  
Press ENTER to exit console. 
```

RESULT:

Thus the data encryption standard algorithm had been implemented successfully using C language.

BLOWFISH

AIM

To implement the **Blowfish symmetric encryption algorithm** for encrypting and decrypting a given plaintext using a secret key.

ALGORITHM

1. Start the program.
2. Read the plaintext and secret key from the user.
3. Create a Blowfish cipher using the given key.
4. Encrypt the plaintext and display the ciphertext.
5. Initialize the cipher in decryption mode using the same key.
6. Decrypt the ciphertext.
7. Display the original plaintext.
8. Stop the program.

PROGRAM

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;

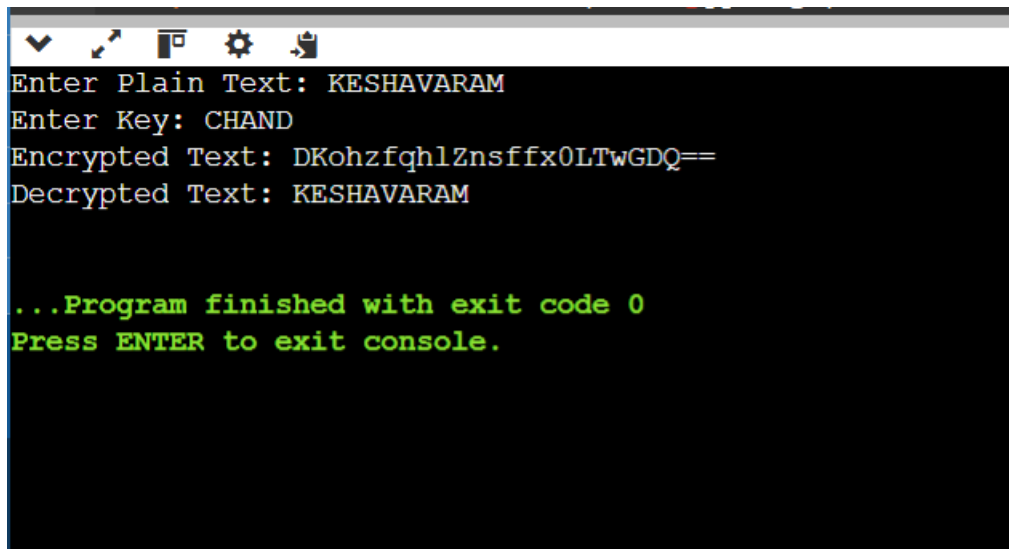
public class Main {
    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.nextLine();

        System.out.print("Enter Key: ");
        String key = sc.nextLine();
```

```
SecretKeySpec secretKey =  
    new SecretKeySpec(key.getBytes(), "Blowfish");  
  
Cipher cipher = Cipher.getInstance("Blowfish");  
  
cipher.init(Cipher.ENCRYPT_MODE, secretKey);  
byte[] encrypted = cipher.doFinal(text.getBytes());  
  
System.out.println("Encrypted Text: " +  
    Base64.getEncoder().encodeToString(encrypted));  
  
cipher.init(Cipher.DECRYPT_MODE, secretKey);  
byte[] decrypted = cipher.doFinal(encrypted);  
  
System.out.println("Decrypted Text: " +  
    new String(decrypted));  
  
sc.close();  
}  
}
```



```
Enter Plain Text: KESHAVARAM
Enter Key: CHAND
Encrypted Text: DKohzfqhlZnsffx0LTwGDQ==
Decrypted Text: KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.
```

RESULT

Thus, the **Blowfish encryption and decryption algorithm** was implemented successfully, and the original plaintext was recovered after decryption.

IMPELEMENTATION OF THE RC5

AIM

To implement the RC5 encryption and decryption algorithm for securing a given plaintext message using a secret key.

ALGORITHM

1. Start the program.
2. Read the plaintext and secret key.
3. Generate the RC5 subkeys using the key expansion process.
4. Divide the plaintext into two blocks, A and B.
5. Perform RC5 encryption using XOR, addition, and circular left rotation operations.
6. Display the encrypted ciphertext.
7. Perform RC5 decryption using the subkeys in reverse order.
8. Display the decrypted plaintext.
9. Stop the program.

PROGRAM :

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
#define R 12
```

```
#define T (2 * (R + 1))
```

```
#define W 32
```

```
#define P 0xB7E15163
```

```
#define Q 0x9E3779B9
```

```
uint32_t S[T];
```

```

uint32_t rol(uint32_t x, uint32_t y) {
    y &= 31;
    return (x << y) | (x >> (32 - y));
}

```

```

uint32_t ror(uint32_t x, uint32_t y) {
    y &= 31;
    return (x >> y) | (x << (32 - y));
}

```

```

void keyExpansion(uint8_t *key, int b) {
    uint32_t L[4] = {0};
    uint32_t A, B;
    int i, j, k;

```

```

    for (i = b - 1; i >= 0; i--)
        L[i / 4] = (L[i / 4] << 8) + key[i];

```

```

    S[0] = P;

```

```

    for (i = 1; i < T; i++)
        S[i] = S[i - 1] + Q;

```

```

    A = B = 0;

```

```

    i = j = 0;

```

```

    for (k = 0; k < 3 * T; k++) {

```



```

    A = S[i] = rol(S[i] + A + B, 3);
    B = L[j] = rol(L[j] + A + B, A + B);
    i = (i + 1) % T;
    j = (j + 1) % 4;
}
}

```

```

void encrypt(uint32_t *A, uint32_t *B) {
    int i;

    *A += S[0];
    *B += S[1];

    for (i = 1; i <= R; i++) {
        *A = rol(*A ^ *B, *B) + S[2 * i];
        *B = rol(*B ^ *A, *A) + S[2 * i + 1];
    }
}

```

```

void decrypt(uint32_t *A, uint32_t *B) {
    int i;

    for (i = R; i >= 1; i--) {
        *B = ror(*B - S[2 * i + 1], *A) ^ *A;
        *A = ror(*A - S[2 * i], *B) ^ *B;
    }

    *B -= S[1];
}

```

```
*A -= S[0];  
}
```

```
int main() {  
    char plaintext[9];  
    uint8_t key[16];  
    uint32_t A, B;  
    int i;  
  
    printf("Enter Plain Text (8 characters): ");  
    scanf("%8s", plaintext);  
  
    printf("Enter Key (16 characters): ");  
    scanf("%16s", key);  
  
    keyExpansion(key, 16);  
  
    A = ((uint32_t)(unsigned char)plaintext[0]) |  
        ((uint32_t)(unsigned char)plaintext[1] << 8) |  
        ((uint32_t)(unsigned char)plaintext[2] << 16) |  
        ((uint32_t)(unsigned char)plaintext[3] << 24);  
  
    B = ((uint32_t)(unsigned char)plaintext[4]) |  
        ((uint32_t)(unsigned char)plaintext[5] << 8) |  
        ((uint32_t)(unsigned char)plaintext[6] << 16) |  
        ((uint32_t)(unsigned char)plaintext[7] << 24);  
  
    encrypt(&A, &B);  
}
```

```
printf("\nEncrypted Text: %08X%08X", A, B);
```

```
decrypt(&A, &B);
```

```
char decrypted[9];
```

```
decrypted[0] = A & 0xFF;
```

```
decrypted[1] = (A >> 8) & 0xFF;
```

```
decrypted[2] = (A >> 16) & 0xFF;
```

```
decrypted[3] = (A >> 24) & 0xFF;
```

```
decrypted[4] = B & 0xFF;
```

```
decrypted[5] = (B >> 8) & 0xFF;
```

```
decrypted[6] = (B >> 16) & 0xFF;
```

```
decrypted[7] = (B >> 24) & 0xFF;
```

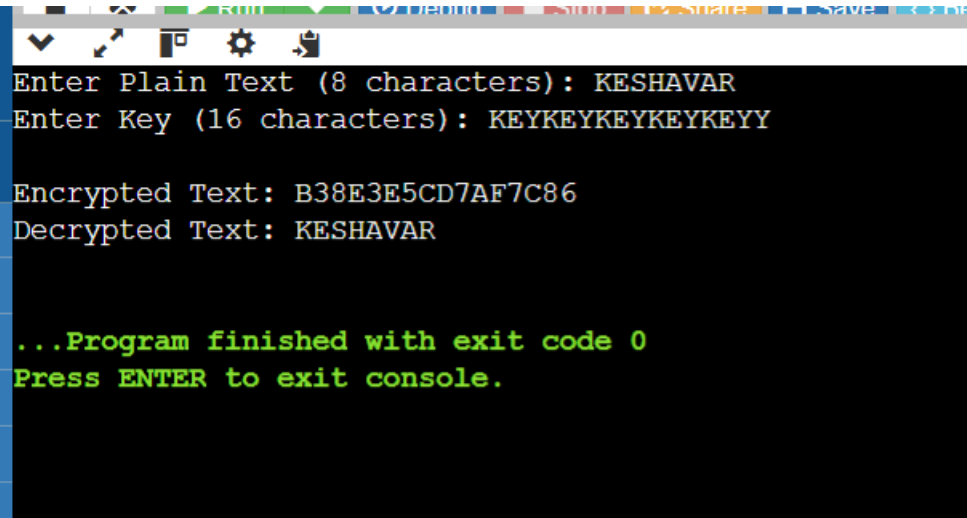
```
decrypted[8] = '\0';
```

```
printf("\nDecrypted Text: %s\n", decrypted);
```

```
return 0;
```

```
}
```

OUTPUT

A screenshot of a console window with a black background and white text. The window has a standard Windows title bar at the top with icons for minimize, maximize, and close, and a menu bar with options like File, Edit, and Help. The text in the console shows the program's execution: it prompts for plain text and a key, performs encryption, and then decryption, successfully recovering the original text. The program ends with a green message indicating it finished with exit code 0.

```
Enter Plain Text (8 characters): KESHAVAR
Enter Key (16 characters): KEYKEYKEYKEYKEYYY
Encrypted Text: B38E3E5CD7AF7C86
Decrypted Text: KESHAVAR

...Program finished with exit code 0
Press ENTER to exit console.
```

RESULT

Thus, the RC5 encryption and decryption algorithm was implemented successfully, and the original plaintext was recovered after decryption.

IMPLEMENTATION OF THE MODE OF DES-ECB,CBC.CFB,OFB

AIM

To implement the four modes of **DES (Data Encryption Standard)**—**ECB, CBC, CFB, and OFB**—for encrypting and decrypting a given plaintext using a secret key.

ALGORITHM

1. Start the program.
2. Read the plaintext and 8-byte DES key.
3. Select the required DES mode: **ECB, CBC, CFB, or OFB**.
4. Initialize the DES cipher with the selected mode and key.
5. Encrypt the plaintext and display the ciphertext.
6. Initialize the cipher in decryption mode using the same key and IV where required.
7. Decrypt the ciphertext and display the original plaintext.
8. Repeat the process for all four DES modes.
9. Stop the program.

PROGRAM:

(FOR ECB)

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.nextLine();

        System.out.print("Enter Key (8 characters): ");
```

```

String key = sc.nextLine();

SecretKeySpec skey = new SecretKeySpec(key.getBytes(), "DES");
Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");

cipher.init(Cipher.ENCRYPT_MODE, skey);
byte[] encrypted = cipher.doFinal(text.getBytes());

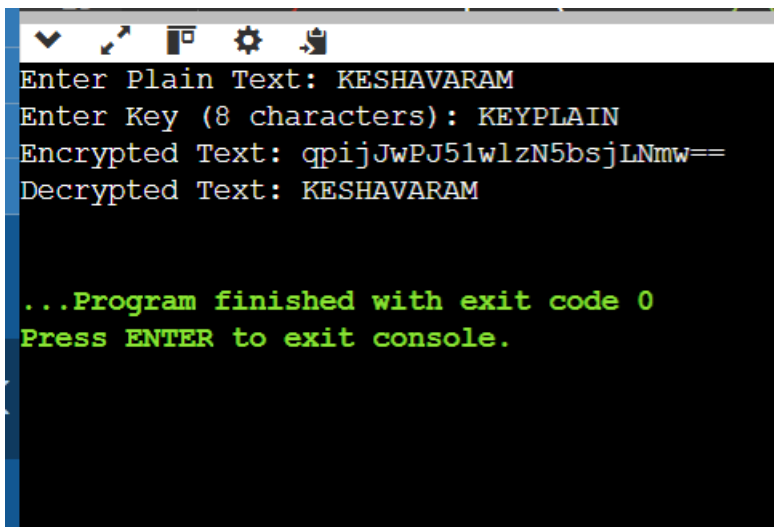
System.out.println("Encrypted Text: " +
    Base64.getEncoder().encodeToString(encrypted));

cipher.init(Cipher.DECRYPT_MODE, skey);
byte[] decrypted = cipher.doFinal(encrypted);

System.out.println("Decrypted Text: " + new String(decrypted));

sc.close();
}
}

```



```

Enter Plain Text: KESHAVARAM
Enter Key (8 characters): KEYPLAIN
Encrypted Text: qpijJwPJ51wlzN5bsjLNmw==
Decrypted Text: KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.

```

PROGRAM:

(CBC)

```
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.nextLine();

        System.out.print("Enter Key (8 characters): ");
        String key = sc.nextLine();

        SecretKeySpec skey = new SecretKeySpec(key.getBytes(), "DES");
        IvParameterSpec iv = new IvParameterSpec("12345678".getBytes());

        Cipher cipher = Cipher.getInstance("DES/CBC/PKCS5Padding");

        cipher.init(Cipher.ENCRYPT_MODE, skey, iv);
        byte[] encrypted = cipher.doFinal(text.getBytes());

        System.out.println("Encrypted Text: " +
            Base64.getEncoder().encodeToString(encrypted));
    }
}
```

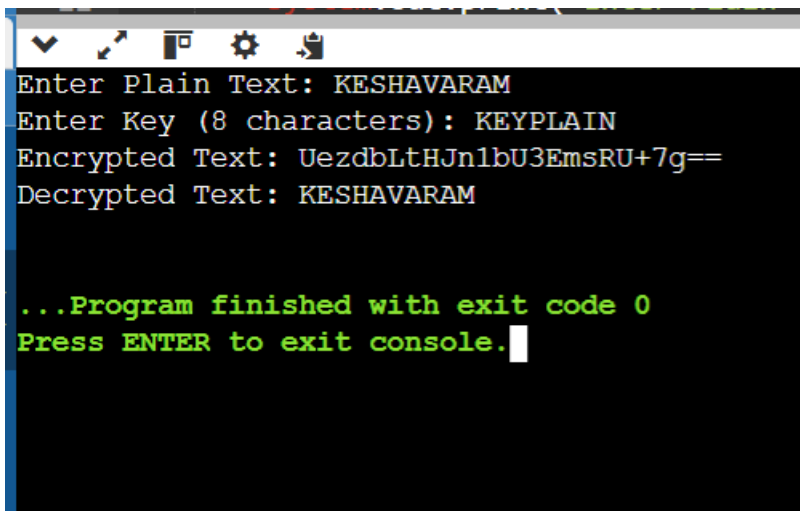
```

cipher.init(Cipher.DECRYPT_MODE, skey, iv);
byte[] decrypted = cipher.doFinal(encrypted);

System.out.println("Decrypted Text: " + new String(decrypted));

sc.close();
}
}

```



```

Enter Plain Text: KESHAVARAM
Enter Key (8 characters): KEYPLAIN
Encrypted Text: UezdbLtHJn1bU3EmsRU+7g==
Decrypted Text: KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.

```

PROGRAM:

(CFB)

```

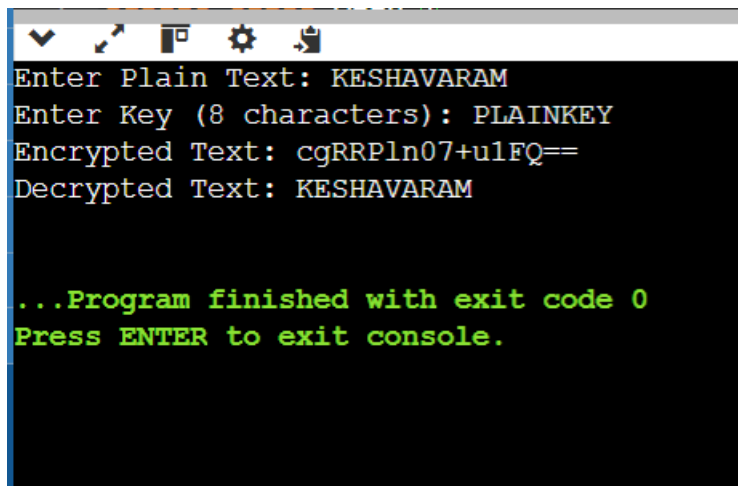
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class Main {

```



```
public static void main(String[] args) throws Exception {  
    Scanner sc = new Scanner(System.in);  
  
    System.out.print("Enter Plain Text: ");  
    String text = sc.nextLine();  
  
    System.out.print("Enter Key (8 characters): ");  
    String key = sc.nextLine();  
  
    SecretKeySpec skey = new SecretKeySpec(key.getBytes(), "DES");  
    IvParameterSpec iv = new IvParameterSpec("12345678".getBytes());  
  
    Cipher cipher = Cipher.getInstance("DES/CFB/NoPadding");  
  
    cipher.init(Cipher.ENCRYPT_MODE, skey, iv);  
    byte[] encrypted = cipher.doFinal(text.getBytes());  
  
    System.out.println("Encrypted Text: " +  
        Base64.getEncoder().encodeToString(encrypted));  
  
    cipher.init(Cipher.DECRYPT_MODE, skey, iv);  
    byte[] decrypted = cipher.doFinal(encrypted);  
  
    System.out.println("Decrypted Text: " + new String(decrypted));  
  
    sc.close();  
}  
}
```



```
Enter Plain Text: KESHAVARAM
Enter Key (8 characters): PLAINKEY
Encrypted Text: cgRRPln07+u1FQ==
Decrypted Text: KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.
```

PROGRAM:

(OFB)

```
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");
        String text = sc.nextLine();

        System.out.print("Enter Key (8 characters): ");
        String key = sc.nextLine();

        SecretKeySpec skey = new SecretKeySpec(key.getBytes(), "DES");
        IvParameterSpec iv = new IvParameterSpec("12345678".getBytes());
```

```
Cipher cipher = Cipher.getInstance("DES/OFB/NoPadding");

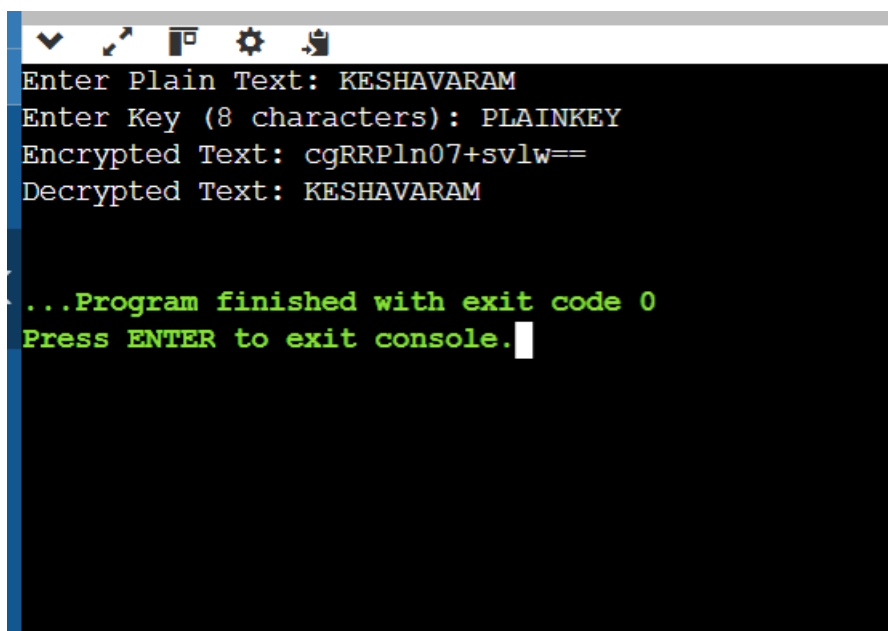
cipher.init(Cipher.ENCRYPT_MODE, skey, iv);
byte[] encrypted = cipher.doFinal(text.getBytes());

System.out.println("Encrypted Text: " +
    Base64.getEncoder().encodeToString(encrypted));

cipher.init(Cipher.DECRYPT_MODE, skey, iv);
byte[] decrypted = cipher.doFinal(encrypted);

System.out.println("Decrypted Text: " + new String(decrypted));

sc.close();
}
}
```

A screenshot of a Java IDE's console window. The window has a dark background with a toolbar at the top containing icons for a dropdown menu, a cursor, a window, a gear, and a document. The console output is as follows:
Enter Plain Text: KESHAVARAM
Enter Key (8 characters): PLAINKEY
Encrypted Text: cgRRPln07+svlw==
Decrypted Text: KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.
The text is displayed in a monospaced font, with the first four lines in white and the last two lines in green.

RESULT

Thus, the **four DES modes—ECB, CBC, CFB, and OFB—were implemented successfully**, and the original plaintext was recovered correctly after decryption.

IMPLEMENTATION OF 3DES

AIM

To implement the **3DES (Triple Data Encryption Standard)** technique for encrypting and decrypting a given message using three keys.

ALGORITHM

1. Start the program.
2. Read the plaintext and three keys from the user.
3. Encrypt the plaintext using **Key 1**.
4. Encrypt the result using **Key 2**.
5. Encrypt the result again using **Key 3**.
6. Display the encrypted text.
7. Decrypt using **Key 3**, **Key 2**, and **Key 1** in reverse order.
8. Display the decrypted plaintext.
9. Stop the program.

PROGRAM

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char text[100];
```

```
    char key1[9], key2[9], key3[9];
```

```
    char encrypted[100], decrypted[100];
```

```
    int i, len;
```

```
    printf("Enter Plain Text: ");
```

```
    fgets(text, sizeof(text), stdin);
```

```
    text[strcspn(text, "\n")] = '\0';
```

```
    printf("Enter Key 1 (8 characters): ");
```

```
scanf("%8s", key1);
```

```
printf("Enter Key 2 (8 characters): ");
```

```
scanf("%8s", key2);
```

```
printf("Enter Key 3 (8 characters): ");
```

```
scanf("%8s", key3);
```

```
len = strlen(text);
```

```
for (i = 0; i < len; i++) {
```

```
    encrypted[i] = text[i] ^ key1[i % 8];
```

```
    encrypted[i] = encrypted[i] ^ key2[i % 8];
```

```
    encrypted[i] = encrypted[i] ^ key3[i % 8];
```

```
}
```

```
printf("\nEncrypted Text: ");
```

```
for (i = 0; i < len; i++)
```

```
    printf("%02X", (unsigned char)encrypted[i]);
```

```
for (i = 0; i < len; i++) {
```

```
    decrypted[i] = encrypted[i] ^ key3[i % 8];
```

```
    decrypted[i] = decrypted[i] ^ key2[i % 8];
```

```
    decrypted[i] = decrypted[i] ^ key1[i % 8];
```

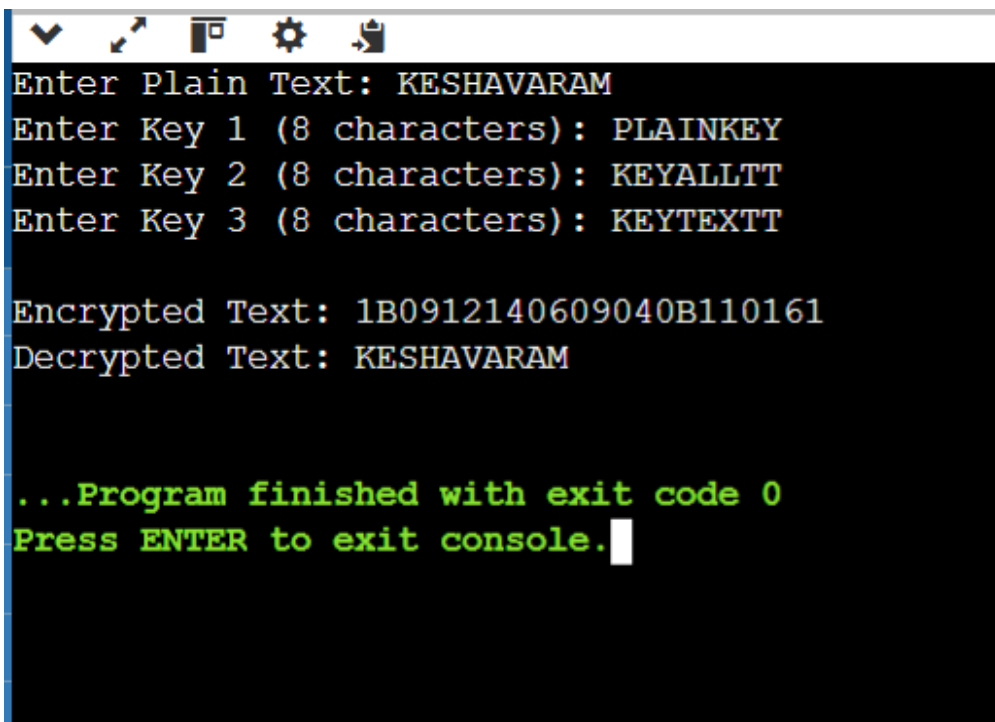
```
}
```

```
decrypted[len] = '\0';
```

```
printf("\nDecrypted Text: %s\n", decrypted);
```

```
    return 0;  
}
```

OUTPUT



```
Enter Plain Text: KESHAVARAM  
Enter Key 1 (8 characters): PLAINKEY  
Enter Key 2 (8 characters): KEYALLTT  
Enter Key 3 (8 characters): KEYTEXTT  
  
Encrypted Text: 1B0912140609040B110161  
Decrypted Text: KESHAVARAM  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

RESULT

Thus, the **3DES encryption and decryption process** was implemented successfully, and the original plaintext was recovered after decryption.

IMPLEMENTATION OF 2DES

AIM

To implement the **2DES (Double Data Encryption Standard)** technique for encrypting and decrypting a given message using two keys.

ALGORITHM

1. Start the program.
2. Read the plaintext and two keys from the user.
3. Encrypt the plaintext using **Key 1**.
4. Encrypt the result again using **Key 2**.
5. Display the encrypted text.
6. Decrypt the encrypted text using **Key 2**.
7. Decrypt the result using **Key 1**.
8. Display the original decrypted text.
9. Stop the program.

PROGRAM :

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char text[100];
```

```
    char key1[9], key2[9];
```

```
    char encrypted[100], decrypted[100];
```

```
    int i, len;
```

```
    printf("Enter Plain Text: ");
```

```
    fgets(text, sizeof(text), stdin);
```

```
    text[strcspn(text, "\n")] = '\0';
```

```
    printf("Enter Key 1 (8 characters): ");
```

```
scanf("%8s", key1);
```

```
printf("Enter Key 2 (8 characters): ");
```

```
scanf("%8s", key2);
```

```
len = strlen(text);
```

```
for (i = 0; i < len; i++) {
```

```
    encrypted[i] = text[i] ^ key1[i % 8];
```

```
    encrypted[i] = encrypted[i] ^ key2[i % 8];
```

```
}
```

```
encrypted[len] = '\0';
```

```
printf("\nEncrypted Text: ");
```

```
for (i = 0; i < len; i++)
```

```
    printf("%02X", (unsigned char)encrypted[i]);
```

```
for (i = 0; i < len; i++) {
```

```
    decrypted[i] = encrypted[i] ^ key2[i % 8];
```

```
    decrypted[i] = decrypted[i] ^ key1[i % 8];
```

```
}
```

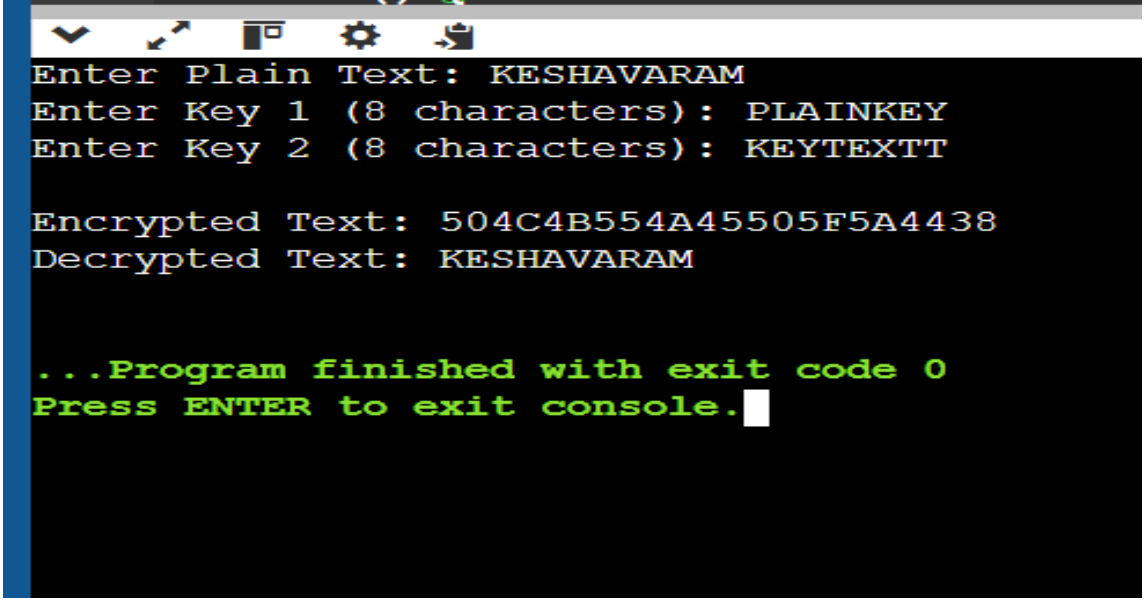
```
decrypted[len] = '\0';
```

```
printf("\nDecrypted Text: %s\n", decrypted);
```

```
return 0;
```

```
}
```

OUTPUT:

A screenshot of a Windows command prompt window. The window has a title bar with standard icons (checkmark, zoom, window, settings, and a document). The background is black, and the text is displayed in a monospaced font. The text is as follows:
Enter Plain Text: KESHAVARAM
Enter Key 1 (8 characters): PLAINKEY
Enter Key 2 (8 characters): KEYTEXTT

Encrypted Text: 504C4B554A45505F5A4438
Decrypted Text: KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.
The text is color-coded: the first three lines are in white, the next two are in yellow, and the last two are in green. A white cursor is visible at the end of the last line.

```
Enter Plain Text: KESHAVARAM
Enter Key 1 (8 characters): PLAINKEY
Enter Key 2 (8 characters): KEYTEXTT

Encrypted Text: 504C4B554A45505F5A4438
Decrypted Text: KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.
```

RESULT

Thus, the **2DES encryption and decryption process** was implemented successfully, and the original plaintext was recovered after decryption.

EX. NO: 7

IMPLEMENTATION OF RSA

AIM:

To write a C program to implement the RSA encryption algorithm.

ALGORITHM:

STEP-1: Select two co-prime numbers as p and q.

STEP-2: Compute n as the product of p and q.

STEP-3: Compute $(p-1)*(q-1)$ and store it in z.

STEP-4: Select a random prime number e that is less than that of z.

STEP-5: Compute the private key, d as $e * \text{mod-1}(z)$.

STEP-6: The cipher text is computed as $\text{message} * \text{mod } n$. STEP-7: Decryption is done as $\text{cipherdmod } n$.

PROGRAM:

```
#include <stdio.h>
```

```
long long modExp(long long base, long long exp, long long mod)
```

```
{
```

```
    long long result = 1;
```

```
    base = base % mod;
```

```
    while (exp > 0)
```

```
    {
```

```
        if (exp % 2 == 1)
```

```
            result = (result * base) % mod;
```

```
        exp = exp / 2;
```

```
        base = (base * base) % mod;
```

```
    }
```

```
    return result;
}
```

```
int gcd(int a, int b)
{
    while (b != 0)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
int modInverse(int e, int phi)
{
    int d;

    for (d = 1; d < phi; d++)
    {
        if ((e * d) % phi == 1)
            return d;
    }

    return -1;
}
```

```
int isPrime(int n)
{

```

```
if (n < 2)
    return 0;
```

```
for (int i = 2; i * i <= n; i++)
{
    if (n % i == 0)
        return 0;
}
```

```
return 1;
}
```

```
int main()
```

```
{
    int p, q;
```

```
    printf("Enter first prime number: ");
    scanf("%d", &p);
```

```
    printf("Enter second prime number: ");
    scanf("%d", &q);
```

```
    if (!isPrime(p) || !isPrime(q) || p == q)
    {
        printf("Invalid prime numbers.\n");
        return 0;
    }
```

```
    int n = p * q;
```

```
int phi = (p - 1) * (q - 1);
```

```
int e;
```

```
for (e = 2; e < phi; e++)
```

```
{
```

```
    if (gcd(e, phi) == 1)
```

```
        break;
```

```
}
```

```
int d = modInverse(e, phi);
```

```
printf("\nPublic Key = (%d, %d)\n", e, n);
```

```
printf("Private Key = (%d, %d)\n", d, n);
```

```
char msg[100];
```

```
printf("\nEnter message (lowercase): ");
```

```
scanf("%s", msg);
```

```
long long cipher[100];
```

```
printf("\nEncrypted Message:\n");
```

```
int i;
```

```
for (i = 0; msg[i] != '\0'; i++)
```

```
{
```

```
    int m = msg[i];
```



```
    cipher[i] = modExp(m, e, n);
    printf("%lld ", cipher[i]);
}

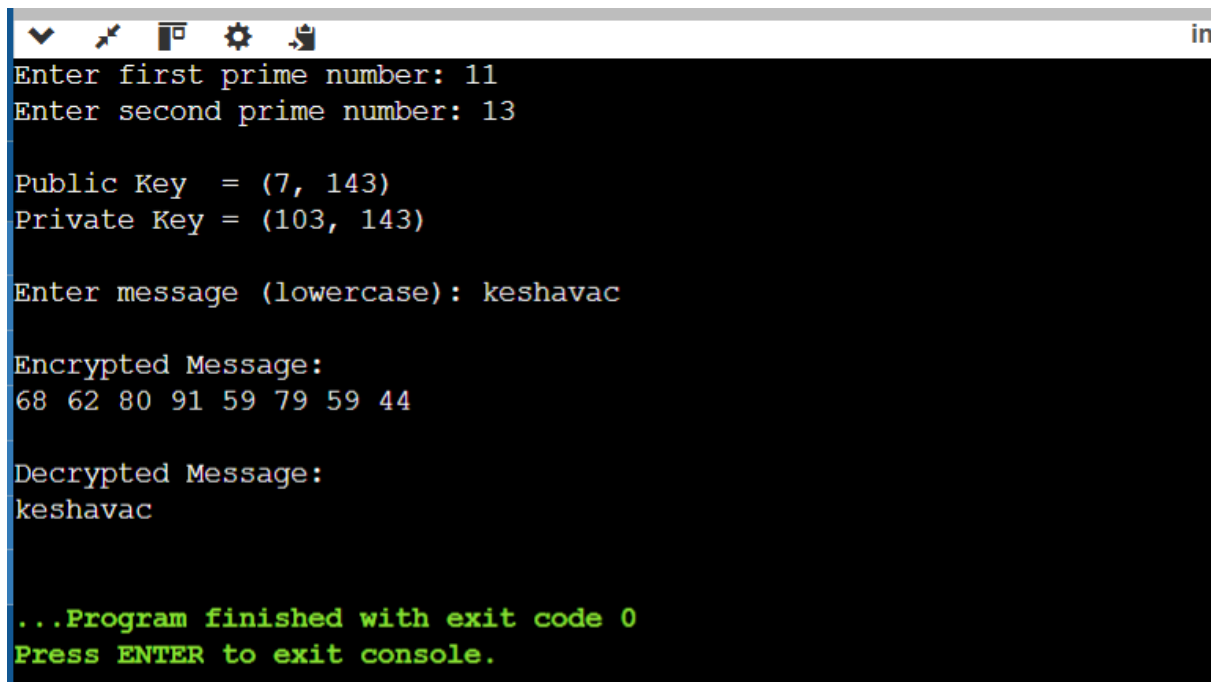
printf("\n\nDecrypted Message:\n");

for (i = 0; msg[i] != '\0'; i++)
{
    char ch = (char)modExp(cipher[i], d, n);
    printf("%c", ch);
}

printf("\n");

return 0;
}
```

OUTPUT:

A screenshot of a terminal window with a dark background and light-colored text. The window has a title bar with standard icons (minimize, maximize, close) and a small 'in' icon on the right. The text inside the terminal shows the execution of an RSA encryption program. It starts with prompts for two prime numbers, 11 and 13. It then displays the calculated public and private keys. Next, it prompts for a message, 'keshavac', and shows the resulting encrypted message as a sequence of numbers. Finally, it shows the decrypted message, 'keshavac', and a green-colored message indicating the program finished successfully.

```
Enter first prime number: 11
Enter second prime number: 13

Public Key = (7, 143)
Private Key = (103, 143)

Enter message (lowercase): keshavac

Encrypted Message:
68 62 80 91 59 79 59 44

Decrypted Message:
keshavac

...Program finished with exit code 0
Press ENTER to exit console.
```

RESULT:

Thus the C program to implement RSA encryption technique had been implemented successfully

